

FastWAM AGX Orin 部署报告

日期：2026-05-08 · 阶段：real_robotwin_text_inference_validation

1. 当前结论

eager .pth baseline 已完成：使用官方 released RoboTwin checkpoint，在 AGX Orin 上通过 PyTorch eager 路径运行。

TensorRT FP16 分区链路已串联 VAE image encoder、video prefill、dynamic-KV action step、host scheduler 和 action denormalize。当前阶段继续验证真实 RoboTwin sample 与真实 FastWAM T5 text cache，不代表 simulator success rate。

```
runtime_mode = eager_pth
backend = pytorch
dtype = bf16
sample_source = synthetic_robotwin_contract
text_context = synthetic_zero_context
semantic_text_encoder = disabled
```

2. 当前 AGX Benchmark

```
warmup / measured = 5 / 20
num_inference_steps = 1
mean_preprocess_ms = 7.57 ms
mean_policy_ms = 317.89 ms
mean_end_to_end_ms = 325.47 ms
p50_end_to_end_ms = 325.51 ms
p95_end_to_end_ms = 326.20 ms
```

当前数值来自 PyTorch eager baseline，不代表 TensorRT 优化后性能，也不代表 RoboTwin simulator success rate。

2.1 真实 RoboTwin Sample 验证

```
sample_source = offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz
instruction = Lift the bottle with narrow top head-up from the table
text_context = precomputed_t5_cache
eager_real_sample_mean_end_to_end = 323.97 ms
partitioned_trt_real_sample_mean_end_to_end = 134.57 ms
partitioned_trt_real_sample_p95 = 134.69 ms
partitioned_trt_output_shape = [1, 32, 14], finite = True
proprio_source = cached_proprio_encoder
```

该验证接入真实 LeRobot/RoboTwin 图像、14D state、checkpoint-derived proprio token，并优先展示真实 T5 text cache 结果；如果 text-cache JSON 缺失，则回退显示 zero-text 历史结果。

2.2 Eager vs TensorRT Drift

```
validation_status = success
validation_mode = eager_fp16_reference_vs_partitioned_trt_fp16_real_robotwin_text_cache
num_compared_tensors = 15
worst_max_abs = 115.62890625
all_compared_finite = True
```

3. TensorRT 子图结果

```
scope = action_step_partition
onnx_export = success, size = 2.37 MiB
fp16_engine = deleted_after_dynamic_kv_replacement, size = 1999.46 MiB, build = 169.619 s
fp16_python_mean_ms = 19.04 ms, fp16_cpp_mean_ms = 19.04 ms
int8_engine = deleted_after_review_unvalidated_probe, size = 3383.15 MiB, build = 628.787 s
int8_python_mean_ms = 33.79 ms, int8_cpp_mean_ms = 33.81 ms
```

```
vae_image_encoder_onnx = deleted_after_engine_build, size = 359.18 MiB
vae_image_encoder_engine = success, size = 303.93 MiB, build = 245.981 s
vae_image_encoder_python_mean_ms = 38.94 ms, p95 = 39.11 ms
```

```
video_prefill_onnx = external_data_deleted_after_engine_build, external_data = 9.1 GiB
video_prefill_engine = success, size = 9263.54 MiB, build = 168.494 s
video_prefill_python_mean_ms = 74.73 ms, p95 = 74.79 ms
video_prefill_outputs = 60, finite = True
```

```
dynamic_kv_action_step_onnx = deleted_after_engine_build, size = 1949.11 MiB, kv_inputs = 60
dynamic_kv_action_step_engine = success, size = 1957.29 MiB, build = 99 s
dynamic_kv_python_mean_ms = 19.03 ms, p95 = 19.05 ms
dynamic_kv_output_shape = [1, 32, 14], finite = True
```

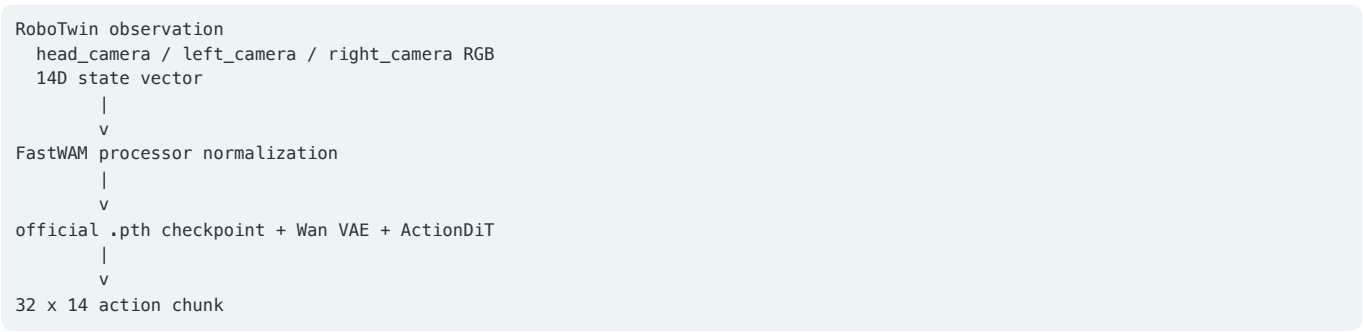
partitioned_runtime = success
partitioned_mean_end_to_end = 134.13 ms, p95 = 134.31 ms
partitioned_vae = 38.95 ms, video_prefill = 74.63 ms, action_loop = 19.33 ms
partitioned_kv_cast = 0.61 ms, scheduler = 0.09 ms, action_decode = 0.21 ms
partitioned_output = [1, 32, 14], finite = True

int8_best_combo = vae_int8_synthcal + video_int8_uncalibrated + action_fp16
int8_best_mean_end_to_end = 121.62 ms, p95 = 121.78 ms, speedup_vs_fp16_partitioned = 1.1
vae_int8_synthcal = success_synthetic_calibration, mean = 26.46 ms
video_int8_synthcal = failed_oom; video_int8_uncal = deleted_after_review_speed_artifact_only, mean = 73.60 ms
action_int8_synthcal = failed_cuda_illegal_memory_access; action_int8_uncal = deleted_after_review_speed_artifact_only, mean = 20.54 ms

4. 工程边界与关键决策

ID	决策	说明
D001	Use a separate deployment scaffold	Deployment code lives in fastwam-deploy. Upstream FastWAM remains a reproducible source dependency and is not modified in place.
D002	Start from eager PyTorch .pth runtime	The first deploy milestone uses the official released RoboTwin checkpoint in PyTorch eager mode. ONNX/TensorRT are deferred until eager is fully documented.
D003	Avoid loading T5 encoder on AGX for the initial baseline	The T5 text encoder is large and risky on Jetson memory/disk. The initial benchmark uses the model context path with synthetic zero context. A precomputed text-cache loader is now implemented for the next real semantic run.
D004	Partition TensorRT conversion instead of converting the full policy first	FastWAM full inference mixes VAE image encoding, video KV prefill, iterative action denoising, scheduler logic, random latent initialization, and Python cache structures. TensorRT work therefore starts from the action denoise-step partition.
D005	Use PyTorch CUDA tensor pointers for the first TensorRT runtime	pycuda and cuda-python are not installed in the current AGX environment. The runtime benchmark binds torch CUDA tensor device pointers directly to TensorRT 10.3 execute_async_v3, avoiding extra dependency changes.
D006	Freeze full runtime partitions before further TensorRT conversion	Phase 1 now profiles the eager runtime by preprocess, context/proprio, VAE first-frame image encode, video pre_dit, video KV prefill, action denoise loop, scheduler step, and action denormalize. This JSON profile is the interface contract for the next TensorRT partitions.
D007	Use deploy-side ONNX-safe VAE patches for TensorRT export	The VAE/image encoder TensorRT export keeps upstream FastWAM unchanged and patches only the deploy export path: SDPA attention becomes explicit matmul/softmax/matmul, RMSNorm becomes explicit RMS arithmetic, CausalConv3d uses static temporal zero concat plus Conv3d spatial padding, and the wrapper calls vae.model.encode directly.
D008	Export video prefill as explicit K/V cache outputs	The video prefill partition takes first_frame_latents, context, and context_mask as inputs and exports 60 explicit outputs: video_k_i and video_v_i for 30 MoT layers. The video-only attention mask is exported as a static all-True mask for the one-frame RoboTwin prefill path.
D009	Use dynamic K/V inputs for the deploy action-step engine	The deployable action-step TensorRT engine now accepts latents_action, timestep_action, context, context_mask, and 60 video_k_i/video_v_i tensors. This avoids baking a synthetic video cache into the engine and makes it compatible with the video prefill engine output contract.
D010	Keep scheduler and action denormalize host-side in the first partitioned TensorRT runtime	The integrated FP16 runtime loads VAE image encoder, video prefill, and dynamic-KV action-step TensorRT engines in one process. Scheduler update and z-score action denormalization stay as small host/PyTorch CUDA tensor operations because their measured cost is below 0.3 ms for the one-step baseline.
D011	Separate accuracy-valid INT8 from speed artifacts	VAE INT8 was built with a synthetic torch-backed calibrator and is useful for runtime engineering but still not accuracy-valid. Dynamic action-step and video prefill calibrated INT8 failed, so their uncalibrated trtexc INT8 engines are recorded only as speed artifacts and must not be treated as deployable quantized models.
D012	Validate real RoboTwin sample before real text-cache work	The real-input milestone first connects a minimal LeRobot/RoboTwin unified sample into eager and partitioned TensorRT runtime. This validates image/state/sample adapter correctness without downloading the large T5 encoder on AGX; semantic text remains blocked until a precomputed FastWAM text cache is supplied.
D013	Use precomputed FastWAM T5 cache for semantic RoboTwin inference	The AGX runtime should not load the 10GB+ T5 encoder during normal inference. The semantic path uses upstream FastWAM precompute_text_embeds.py to produce one prompt-specific context/mask .pt cache, then eager and partitioned TensorRT consume that cache through --text-cache-dir.
D014	Treat drift validation as a gate before simulator claims	The real-text TensorRT chain is latency-valid and finite-output-valid, but numerical drift is not yet small enough to claim accuracy equivalence. The report therefore separates runtime completion from simulator/accuracy readiness.

5. Eager Runtime 链路



当前已新增两个真实输入接口：

- sample: 读取离线 .npz 或 .json RoboTwin 样本。
- text-cache-dir: 读取上游 precompute_text_embeds.py 生成的 T5 context cache。

6. TensorRT Runtime 链路



```
      v
TensorRT FP16 engine
      |
      v
pred_action [1, 32, 14]
```

当前 TensorRT runtime 用 PyTorch CUDA tensor 的 device pointer 直接绑定 TensorRT I/O，不依赖缺失的 pycuda/cuda-python。该实验验证 engine 执行和单步延迟，但尚未替换完整 eager policy。

最新 dynamic-KV action-step engine 已不再内嵌 synthetic video cache，而是显式接收 60 个 video K/V tensor，可作为 VAE/image encoder 和 video prefill engine 之后的集成节点。

当前 partitioned runtime 已在同一进程中串联 VAE、video prefill、dynamic-KV action-step、host scheduler 和 z-score action denormalize。该结果仍使用 synthetic tensor contract，不代表真实 RoboTwin 语义样本或 simulator success rate。

INT8 阶段目前只有 VAE synthetic-calibrated engine 有明确收益；video/action 的 calibrated INT8 分别受 OOM 和 CUDA illegal memory access 阻塞，uncalibrated INT8 仅作为速度 artifact 记录。

7. Full Eager Partition Profile

```
runtime_mode = eager_ptn_partition_profile
num_inference_steps = 1
mean_end_to_end = 298.98 ms
preprocess = 7.49 ms
vae_image_encode = 55.26 ms
video_pre_dit = 2.57 ms
video_prefill_cache = 115.37 ms
action_denoise_loop = 115.79 ms
action_step_mot_with_cache = 113.97 ms
action_denormalize = 0.47 ms
```

该 profile 是后续把 VAE/image prefill/scheduler/action decode 分阶段替换成 TensorRT/CUDA 的接口冻结依据。

5. Issues / 处理记录

已记录 45 个部署问题，其中 resolved=33，open=12。

ID	严重度	组件	现象	处理/方案	状态
I001	high	storage	AGX root filesystem is full, with roughly 47 MB free.	Created /root/wujinlong/tinyaction_fastwam as a symlink to /root/wujinlong/nvme/tinyaction_fastwam. All deploy artifacts, checkpoints, caches, and outputs are kept under the NVMe-backed project path.	resolved
I002	high	dependency	FastWAM official dependency stack requests x86 CUDA wheels such as torch==2.7.1+cu128, which are unsuitable for Jetson.	Reused /root/wujinlong/pi0.5/.venv with torch 2.8.0 CUDA available on Orin. Additional Python packages are installed into python_deps with --no-deps to avoid shadowing Jetson torch.	resolved
I003	medium	dependency	An early dependency install into python_deps pulled a torch 2.11 CUDA 13 package.	Removed the accidental torch package from python_deps and reinstalled lightweight dependencies with --no-deps. Verified torch imports from /root/wujinlong/pi0.5/.venv.	resolved
I004	medium	reporting	Initial benchmark JSON wrote checkpoint and dataset_stats as null.	Added resolve_runtime_paths usage in eval_open_loop_fastwam.py and benchmark_fastwam.py so JSON records the actual default checkpoint and stats paths.	resolved
I005	medium	benchmarking	FastWAM mean_policy_ms initially included image/state preprocessing, making it inconsistent with pi0.5 benchmark semantics.	Split preprocessing and policy execution into separate timed stages. mean_preprocess_ms, mean_policy_ms, and mean_end_to_end_ms now follow the pi0.5 style.	resolved
I006	medium	sync	An rsync --delete pass removed the AGX outputs/ benchmark JSON because outputs were not present locally.	Added .gitignore and switched sync usage to exclude outputs/, artifacts/, data/, tmp/, hf_home/, and cache/. Re-ran the formal benchmark to restore the AGX JSON result.	resolved
I007	medium	data	AGX project path currently has no robotwin2.0 dataset or text_embeds_cache directory.	Implemented offline sample adapter and precomputed text-cache loader. Real sample/cache benchmark is blocked until those assets are copied or generated.	resolved
I008	medium	tensorrt	AGX TensorRT probe found TensorRT 10.3/trtexec and Python tensorrt binding available, but onnx_graphsurgeon, polygraphy, pycuda, and cuda-python are missing.	Record as TensorRT-stage dependency gap. For first engine build use trtexec and TensorRT Python APIs already present. Install graph surgery/runtime helpers only if required, under NVMe-backed project path and without replacing Jetson torch.	open
I009	high	tensorrt	FastWAM eager inference is not a single static feed-forward graph.	Do not attempt full-model TensorRT conversion first. Start with subgraph partitioning: video prefill/cache and action denoise step. Probe ONNX export for the action denoise step before building FP16/INT8 engines.	open
I010	medium	onnx_export	The first FP16 action-step ONNX export on AGX reached eager output shape [1, 32, 14], but failed during legacy ONNX constant folding with a mixed cuda:0/cpu tensor-device error.	Changed export_action_step_onnx.py to disable PyTorch ONNX constant folding by default. TensorRT builder will perform its own graph optimizations after a valid ONNX is produced.	resolved
I011	medium	onnx_export	Legacy ONNX export of the action-step probe previously failed with ScalarType ComplexDouble because FastWAM RoPE uses torch.view_as_complex/view_as_real.	Installed a deploy-side monkey patch that replaces FastWAM's complex RoPE helper with a real-valued cos/sin implementation during ONNX export only. Upstream FastWAM source remains unchanged.	resolved
I012	low	tensorrt_runtime	The first TensorRT runtime benchmark script failed before engine execution with ValueError: Expected a torch.device with a specified index or an integer, but got:cuda.	Changed benchmark_trt_action_step.py default device to cuda:0 and normalized unspecified CUDA devices to index 0 before binding TensorRT I/O pointers.	resolved
I013	low	tensorrt_runtime	The TensorRT runtime benchmark script failed with RuntimeError: TensorRT context shape inference failed.	Updated benchmark_trt_action_step.py to explicitly set static input shapes and treat a non-empty infer_shapes() result as the failure condition.	resolved
I014	low	tensorrt_runtime	The first successful TensorRT runtime benchmark emitted a warning that using the default stream in enqueueV3 may cause extra synchronization and lower performance.	Changed the benchmark to create and use a dedicated torch.cuda.Stream for TensorRT execute_async_v3 timing.	resolved
I015	medium	tensorrt_engine	FP16 TensorRT engine build succeeded, but trtexec warned that	Recorded as an accuracy caveat for the FP16 action-step engine. The first runtime milestone keeps the successful	open

ID	严重度	组件	现象	处理/方案	状态
			layernorm nodes in FP16 with Reduce/Pow may overflow.	engine; future accuracy validation should compare TensorRT action-step output against PyTorch eager and may force selected normalization layers to FP32 or export a normalization-friendly graph.	
I016	medium	tensorrt_int8	INT8 trtexec build emits: Calibrator is not being used. Users must provide dynamic range for all tensors that are not Int32 or Bool.	Treat any resulting INT8 engine as a speed/profiling artifact only, not an accuracy-valid deployment engine. A production INT8 path needs representative RoboTwin samples and explicit calibration/dynamic-range generation.	open
I017	medium	cpp_cuda_runtime	AGX has TensorRT/CUDA runtime libraries and trtexec, but C++ headers such as NvInfer.h and cuda_runtime_api.h were not installed under the root filesystem.	Downloaded the required .deb packages and extracted them into /root/wujinlong/tinyaction_fastwam/deps/sysroot on NVMe. The C++ build script uses this sysroot for headers while linking against the already installed runtime libraries.	resolved
I018	low	cpp_cuda_runtime	The first C++ compile attempt failed with fatal error: crt/host_defines.h: No such file or directory.	Downloaded cuda-crt-12-6 and extracted it into the NVMe sysroot. The build script now includes the sysroot CUDA include directories.	resolved
I019	low	cpp_cuda_runtime	The second C++ compile reached link stage but failed with cannot find -lvinfer and cannot find -lcudart.	Changed scripts/build_cpp_trt_runtime.sh to link directly against /usr/lib/aarch64-linux-gnu/libnvinfer.so.10 and /usr/local/cuda/lib64/libcudart.so.12 while still using the NVMe sysroot only for headers.	resolved
I020	medium	profiling	Full eager partition profiling shows peak CUDA allocation around 12.79 GiB even for batch=1 and num_inference_steps=1.	Keep the TensorRT plan partitioned. Do not load the T5 text encoder on AGX for this baseline; use precomputed text cache when available. Build and benchmark one partition at a time and monitor memory before integrating multiple engines.	open
I021	low	profiling	Phase 1 profiling still uses synthetic RoboTwin sample and synthetic zero text context.	The deploy repo already supports --sample and --text-cache-dir. Current TensorRT partition work can continue because tensor shapes match the FastWAM RoboTwin contract, but semantic evaluation must be rerun once real sample/cache assets are copied.	open
I022	medium	vae_onnx_tensorrt	The first VAE image encoder ONNX parsed until /encoder/middle.1/If, then TensorRT failed because the If branches had shapes [1,480,640] and [1,1,480,640].	Added a deploy-side safe attention patch in export_vae_image_encoder_onnx.py that replaces SDPA with explicit matmul, softmax, and matmul for the fixed one-head VAE attention path.	resolved
I023	medium	vae_onnx_tensorrt	After the attention patch, TensorRT failed on a top-level If with shapes [48,1,24,20] and [1,48,1,24,20].	Changed the deploy export wrapper to call vae.model.encode directly on a static [1,3,1,384,320] video tensor, preserving the desired output shape [1,48,1,24,20] without exporting the list wrapper.	resolved
I024	medium	vae_onnx_tensorrt	With If nodes removed, TensorRT parsed the ONNX but engine build failed with dimsUtils.h assertion: Cannot add shapes of unequal rank.	Added deploy-side primitive patches: RMSNorm is exported as explicit mean-square rsqrt arithmetic, and CausalConv3d is exported as static temporal zero concat plus Conv3d spatial padding. The patched ONNX built successfully into a FP16 TensorRT engine.	resolved
I025	medium	vae_tensorrt_engine	VAE FP16 TensorRT build emits a layernorm warning about FP16 Reduce/Pow overflow risk.	Record as an accuracy caveat. The current milestone validates runtime shape/finite output and latency; later accuracy validation should compare TensorRT first_frame_latents against eager output on representative samples and force sensitive normalization operations to FP32 if needed.	open
I026	low	automation	The outer ssh command running trtexec timed out even though the TensorRT log later showed engine generation completed successfully.	Checked pgrep, engine file size, and trtexec log directly on AGX, then ran the separate Python TensorRT benchmark against the generated engine.	resolved
I027	medium	video_prefill_onnx	The first video prefill ONNX export failed inside PyTorch ONNX shape inference with an internal index_put assertion: shape_has_zero=true and minus_one_pos=-1.	Avoided exporting the in-place mask construction. For first-frame video-only prefill, the required video self-attention mask is a static all-True [120,120] tensor, which exports cleanly.	resolved
I028	medium	video_prefill_onnx	The video prefill ONNX file is only about 1.62 MiB, but it references 801 external weight files totaling about 9.1 GiB.	Kept the external data files in outputs/trt for TensorRT build. Any sync/backup step must preserve the .onnx file together with the model.video_expert* external weight files.	resolved
I029	high	video_prefill_tensorrt_engine	The video prefill FP16 engine is about 9.1 GiB and TensorRT build peak CPU memory reached about 35.6 GiB.	Validated all three FP16 engines in one AGX process. The partitioned runtime deserialized VAE, video prefill, and dynamic-KV action-step engines together and still reported about 45.75 GiB free CUDA memory after benchmark. Keep monitoring if real text/cache or more engines are added.	resolved

ID	严重程度	组件	现象	处理/方案	状态
I030	medium	video_prefill_tensorrt_runtime	Some video_v_i outputs are TensorRT FLOAT while the corresponding video_k_i outputs are HALF.	The partitioned runtime binds 31 FP16 K/V tensors directly and preallocates 29 FP16 cast buffers for FP32 video_v_i outputs. The measured cast stage costs about 0.61 ms and avoids per-iteration allocation.	resolved
I031	low	automation	The first dynamic-KV action-step TensorRT build wrapper failed before running trtexec with bash: /usr/bin/time: No such file or directory.	Replaced /usr/bin/time with shell date-based start/end timestamps in the build wrapper. The second trtexec build completed successfully and recorded elapsed_sec=99 in the log.	resolved
I032	medium	action_step_dynamic_kv_tensorrt	Dynamic-KV action-step TensorRT build emitted protobuf large-message warnings and FP16 layernorm overflow warnings.	Build and runtime are successful, and pred_action output is finite. Record the warning as an accuracy caveat; later validation should compare the TensorRT dynamic-KV action-step output against PyTorch eager using the same real video K/V cache and context.	open
I033	medium	partitioned_tensorrt_runtime	The initial full partitioned TensorRT runtime used synthetic image/context tensors rather than a real RoboTwin sample and precomputed text embedding cache.	Resolved for offline inference by adding --sample and --text-cache-dir support, generating a real RoboTwin sample, generating the matching T5 cache entry, and rerunning both eager and partitioned TensorRT with precomputed_t5_cache. Simulator success-rate validation remains a separate task.	resolved
I034	low	int8_build	The VAE INT8 synthetic-calibrated build exceeded the foreground SSH timeout even though it eventually completed.	Use background/nohup build wrappers for long TensorRT builds. The VAE INT8 synthetic-calibrated engine completed successfully, produced a 164 MiB engine, and benchmarked at 26.46 ms.	resolved
I035	high	action_step_int8_calibration	Dynamic-KV action-step synthetic-calibrated INT8 build failed during TensorRT calibration with CUDA illegal memory access and context->executeV2 assertion failure.	Do not use the failed calibrated INT8 path. Built an uncalibrated trtexec INT8+FP16 artifact for speed reference only; it runs finite but is slower than FP16, so the deploy path should keep the FP16 dynamic action-step engine until real calibration/export changes are available.	open
I036	high	video_prefill_int8_calibration	Video prefill synthetic-calibrated INT8 build failed with TensorRT out-of-memory after requesting 34,359,738,368 bytes of GPU memory.	Treat calibrated video INT8 as blocked on a lower-memory quantization strategy, graph split, output pruning during calibration, or representative calibration built off-device. The uncalibrated trtexec INT8+FP16 artifact is only a speed reference.	open
I037	medium	storage	After building extra INT8 artifacts, NVMe free space dropped to about 24 GiB and the filesystem reached about 98% usage.	Keep the artifacts for reproducibility for now, but future runs should archive or delete unhelpful uncalibrated INT8 engines before more builds. The uncalibrated action INT8 engine is slower than FP16 and is a candidate for deletion after review.	open
I038	low	real_sample_adapter	The first real RoboTwin sample export failed while serializing metadata_json with TypeError: Object of type ndarray is not JSON serializable.	Added recursive jsonable conversion for numpy arrays, numpy scalar values, lists, tuples, and dicts before writing metadata_json into the .npz sample. Re-ran the exporter successfully.	resolved
I039	medium	text_cache	AGX initially had no FastWAM text_embeds_cache directory, so the first real sample benchmarks used synthetic zero text context.	Generated the real instruction cache with upstream precompute_text_embeds.py and loaded it from data/text_embeds_cache/robotwin. Eager real-text benchmark and partitioned TensorRT real-text benchmark both report text_context=precomputed_t5_cache.	resolved
I040	low	storage_cleanup	AGX NVMe had only about 13 GiB free after preserving FP16/INT8 TensorRT artifacts.	Deleted /root/wujinlong/tinyaction_fastwam/fastwam-deploy/outputs/trt/video_prefill_int8_uncal.engine after user approval. NVMe free space increased from about 13 GiB to about 22 GiB.	resolved
I041	low	storage_cleanup	FastWAM deploy outputs still contained about 32.14 GiB of obsolete internal artifacts after the video INT8 artifact cleanup.	Created docs/disk_cleanup_manifest_2026-05-08_fastwam_internal.json, deleted the listed internal artifacts, regenerated proprio_encoder.pt as a 122 KiB cloned tensor cache, and verified the real-sample FP16 partitioned runtime still succeeds with output_finite=true. NVMe free space increased from about 22 GiB to about 54 GiB.	resolved
I042	medium	text_cache_download	DiffSynth-Studio/Wan-Series-Converted-Safetensors returned 401 and Hugging Face Xet downloads stalled while preparing the text encoder asset.	Switched the text-cache generation to official Wan-AI/Wan2.2-Ti2V-5B with model.redirect_common_files=false and set HF_HUB_DISABLE_XET=1. The required models_t5_umd5-xxl-enc-bf16.pth file was downloaded and the prompt-specific cache was generated successfully.	resolved
I043	low	validation	Latency-only real-text benchmark cannot show numerical drift between eager PyTorch and the partitioned TensorRT chain.	Added scripts/compare_partitioned_trt_drift.py. It runs an eager FP16 reference and the partitioned FP16 TensorRT runtime on the same RoboTwin sample, T5 cache, seed, initial action latent, and scheduler, then writes compact drift	resolved

ID	严重度	组件	现象	处理/方案	状态
				statistics to outputs/validation/partitioned_trt_drift_real_text_cache.json.	
I044	high	vae_onnx_export	The first drift run showed large first_frame_latents mismatch. Replaying the VAE export-safe patch in PyTorch reproduced the same drift before TensorRT.	Fixed safe_rms_norm_forward to use $x * \text{rsqrt}(\text{mean}(x^2) + \text{eps})$, verified patched PyTorch VAE vs original VAE improved to max_abs about 0.00696 and cosine about 0.999997, then rebuilt vae_image_encoder_fp16_patched.engine.	resolved
I045	high	tensorrt_accuracy	After fixing the RMS export patch and rebuilding VAE FP16 TensorRT, real-text drift still reports first_frame_latents mean_abs about 0.725, selected video K/V max_abs up to about 115.63, and final action_decoded mean_abs about 0.277.	Keep the FP16 TensorRT chain as a runtime benchmark only. Before simulator success-rate claims, test VAE FP32 or selective FP32 normalization/layer precision, and rerun the same drift script until action drift is acceptable.	open

8. 当前 Eager Benchmark JSON

```
{
  "preset": "robotwin_uncond_3cam_384",
  "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
  "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
  "device": "cuda",
  "backend": "pytorch",
  "mixed_precision": "bf16",
  "dtype": "bf16",
  "runtime_mode": "eager_pt",
  "batch_size": 1,
  "sample_source": "synthetic_robotwin_contract",
  "text_context": "synthetic_zero_context",
  "text_cache_path": null,
  "semantic_text_encoder": "disabled",
  "prompt": null,
  "instruction": "Pick up the object and place it at the target.",
  "warmup_batches": 5,
  "measure_batches": 20,
  "num_inference_steps": 1,
  "mean_preprocess_ms": 7.5745124006061815,
  "mean_policy_ms": 317.8902461004327,
  "mean_end_to_end_ms": 325.46709294983884,
  "p50_end_to_end_ms": 325.51256399892736,
  "p95_end_to_end_ms": 326.20384699839633,
  "p50_policy_ms": 317.9273429996101,
  "p95_policy_ms": 318.5825609980384,
  "image_preprocess_ms": 7.004370400682092,
  "state_normalize_ms": 0.46857665220159106,
  "context_ms": 0.005075449735159054,
  "model_forward_ms": 317.42710015241755,
  "action_postprocess_ms": 0.3141636472719256
}
```

9. 真实 Sample Eager Benchmark JSON

```
{
  "preset": "robotwin_uncond_3cam_384",
  "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
  "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
  "device": "cuda",
  "backend": "pytorch",
  "mixed_precision": "bf16",
  "dtype": "bf16",
  "runtime_mode": "eager_pt",
  "batch_size": 1,
  "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
  "text_context": "synthetic_zero_context",
  "text_cache_path": null,
  "semantic_text_encoder": "disabled",
  "prompt": null,
  "instruction": "Lift the bottle with narrow top head-up from the table",
  "warmup_batches": 2,
  "measure_batches": 5,
  "num_inference_steps": 1,
  "mean_preprocess_ms": 7.552995788864791,
  "mean_policy_ms": 323.5628932074178,
  "mean_end_to_end_ms": 331.1181543977,
  "p50_end_to_end_ms": 331.20964499539696,
  "p95_end_to_end_ms": 331.44176402129233,
  "p50_policy_ms": 323.5935270204209,
  "p95_policy_ms": 323.9332170051057,
  "image_preprocess_ms": 6.982205796521157,
  "state_normalize_ms": 0.4662593884859234,
  "context_ms": 0.005144794704392552,
  "model_forward_ms": 323.1001146021299,
  "action_postprocess_ms": 0.3106099306978285
}
```

10. 真实 Sample + Text Cache Eager Benchmark JSON

```
{
  "preset": "robotwin_uncond_3cam_384",
  "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
  "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
  "device": "cuda",
  "backend": "pytorch",
  "mixed_precision": "bf16",
  "dtype": "bf16",
  "runtime_mode": "eager_pt",
  "batch_size": 1,
  "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
  "text_context": "precomputed_t5_cache",
  "text_cache_path": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/text_embeds_cache/robotwin/82243a4317e52ea0809eaaef4ab867833e575409d75cc0eabddd4d352133bde2.t5_len128.wan22ti2v5b.pt",
  "semantic_text_encoder": "precomputed_cache",
  "prompt": "A video recorded from a robot's point of view executing the following instruction: Lift the bottle with narrow top head-up from the table",
  "instruction": "Lift the bottle with narrow top head-up from the table",
}
```

```

"warmup_batches": 2,
"measure_batches": 5,
"num_inference_steps": 1,
"mean_preprocess_ms": 7.464748405618593,
"mean_policy_ms": 316.49949600105174,
"mean_end_to_end_ms": 323.9667852059938,
"p50_end_to_end_ms": 324.02543601347134,
"p95_end_to_end_ms": 324.39730400801636,
"p50_policy_ms": 316.6137360094581,
"p95_policy_ms": 316.8622149969451,
"image_preprocess_ms": 6.9030446000397205,
"state_normalize_ms": 0.45530060306191444,
"context_ms": 0.00522880582138896,
"model_forward_ms": 316.03083219961263,
"action_postprocess_ms": 0.3185819950886071
}

```

11. TensorRT FP16 Runtime JSON

```

{
  "runtime_mode": "tensorrt_action_step",
  "backend": "tensorrt",
  "precision": "fp16_engine",
  "engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_probe_fp16.engine",
  "engine_size_bytes": 2096580572,
  "deserialize_ms": 770.1285619987175,
  "warmup_batches": 20,
  "measure_batches": 100,
  "mean_policy_ms": 19.039098567962647,
  "p50_policy_ms": 19.038976669311523,
  "p95_policy_ms": 19.05561637878418,
  "min_policy_ms": 19.013216018676758,
  "max_policy_ms": 19.063552856445312,
  "output_shape": [
    1,
    32,
    14
  ],
  "output_finite": true,
  "tensors": [
    {
      "index": 0,
      "name": "latents_action",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1,
        32,
        14
      ]
    },
    {
      "index": 1,
      "name": "timestep_action",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1
      ]
    },
    {
      "index": 2,
      "name": "context",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1,
        129,
        4096
      ]
    },
    {
      "index": 3,
      "name": "context_mask",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.BOOL",
      "torch_dtype": "torch.bool",
      "shape": [
        1,
        129
      ]
    },
    {
      "index": 4,
      "name": "pred_action",
      "mode": "TensorIOMode.OUTPUT",
      "trt_dtype": "DataType.FLOAT",
      "torch_dtype": "torch.float32",
      "shape": [
        1,
        32,
        14
      ]
    }
  ]
}

```

```

    ]
  },
  "note": "This benchmark covers only the TensorRT action denoise-step partition, not full FastWAM eager end-to-end inference."
}

```

12. TensorRT INT8 Runtime JSON

```

{
  "runtime_mode": "tensorrt_action_step",
  "backend": "tensorrt",
  "precision": "int8_engine",
  "engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_probe_int8.engine",
  "engine_size_bytes": 3547492996,
  "deserialize_ms": 1896.9985400035512,
  "warmup_batches": 20,
  "measure_batches": 100,
  "mean_policy_ms": 33.791160125732425,
  "p50_policy_ms": 33.790401458740234,
  "p95_policy_ms": 33.80940628051758,
  "min_policy_ms": 33.76259231567383,
  "max_policy_ms": 33.818302154541016,
  "output_shape": [
    1,
    32,
    14
  ],
  "output_finite": true,
  "tensors": [
    {
      "index": 0,
      "name": "latents_action",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1,
        32,
        14
      ]
    },
    {
      "index": 1,
      "name": "timestep_action",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1
      ]
    },
    {
      "index": 2,
      "name": "context",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1,
        129,
        4096
      ]
    },
    {
      "index": 3,
      "name": "context_mask",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.BOOL",
      "torch_dtype": "torch.bool",
      "shape": [
        1,
        129
      ]
    },
    {
      "index": 4,
      "name": "pred_action",
      "mode": "TensorIOMode.OUTPUT",
      "trt_dtype": "DataType.FLOAT",
      "torch_dtype": "torch.float32",
      "shape": [
        1,
        32,
        14
      ]
    }
  ],
  "note": "This benchmark covers only the TensorRT action denoise-step partition, not full FastWAM eager end-to-end inference."
}

```

13. TensorRT C++/CUDA Runtime JSON

```

{
  "runtime_mode": "tensorrt_action_step_cpp_cuda",

```

```

"backend": "tensorrt_cpp_cuda",
"precision": "fp16_engine",
"engine": "outputs/trt/action_step_probe_fp16.engine",
"engine_size_bytes": 2096580572,
"deserialize_ms": 1726.262349,
"warmup_batches": 20,
"measure_batches": 100,
"mean_policy_ms": 19.038055,
"p50_policy_ms": 19.038656,
"p95_policy_ms": 19.056288,
"min_policy_ms": 19.015265,
"max_policy_ms": 19.063072,
"tensors": [
  {
    "name": "latents_action",
    "mode": "input",
    "dtype": "float16",
    "shape": [
      1,
      32,
      14
    ],
    "bytes": 896
  },
  {
    "name": "timestep_action",
    "mode": "input",
    "dtype": "float16",
    "shape": [
      1
    ],
    "bytes": 2
  },
  {
    "name": "context",
    "mode": "input",
    "dtype": "float16",
    "shape": [
      1,
      129,
      4096
    ],
    "bytes": 1056768
  },
  {
    "name": "context_mask",
    "mode": "input",
    "dtype": "bool",
    "shape": [
      1,
      129
    ],
    "bytes": 129
  },
  {
    "name": "pred_action",
    "mode": "output",
    "dtype": "float32",
    "shape": [
      1,
      32,
      14
    ],
    "bytes": 1792
  }
],
"note": "C++ TensorRT/CUDA runtime benchmark for the action denoise-step partition only."
}

```

```

{
  "runtime_mode": "tensorrt_action_step_cpp_cuda",
  "backend": "tensorrt_cpp_cuda",
  "precision": "int8_engine",
  "engine": "outputs/trt/action_step_probe_int8.engine",
  "engine_size_bytes": 3547492996,
  "deserialize_ms": 2436.182309,
  "warmup_batches": 20,
  "measure_batches": 100,
  "mean_policy_ms": 33.806003,
  "p50_policy_ms": 33.8064,
  "p95_policy_ms": 33.825409,
  "min_policy_ms": 33.776577,
  "max_policy_ms": 33.838047,
  "tensors": [
    {
      "name": "latents_action",
      "mode": "input",
      "dtype": "float16",
      "shape": [
        1,
        32,
        14
      ],
      "bytes": 896
    },
    {
      "name": "timestep_action",

```

```

    "mode": "input",
    "dtype": "float16",
    "shape": [
        1
    ],
    "bytes": 2
},
{
    "name": "context",
    "mode": "input",
    "dtype": "float16",
    "shape": [
        1,
        129,
        4096
    ],
    "bytes": 1056768
},
{
    "name": "context_mask",
    "mode": "input",
    "dtype": "bool",
    "shape": [
        1,
        129
    ],
    "bytes": 129
},
{
    "name": "pred_action",
    "mode": "output",
    "dtype": "float32",
    "shape": [
        1,
        32,
        14
    ],
    "bytes": 1792
}
],
"note": "C++ TensorRT/CUDA runtime benchmark for the action denoise-step partition only."
}

```

14. Full Eager Partition Profile JSON

```

{
    "preset": "robotwin_uncond_3cam_384",
    "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
    "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
    "device": "cuda",
    "backend": "pytorch",
    "runtime_mode": "eager_pth_partition_profile",
    "mixed_precision": "bf16",
    "dtype": "bf16",
    "batch_size": 1,
    "sample_source": "synthetic_robotwin_contract",
    "text_context": "synthetic_zero_context",
    "text_cache_path": null,
    "semantic_text_encoder": "disabled",
    "prompt": null,
    "instruction": "Pick up the object and place it at the target.",
    "warmup_batches": 2,
    "measure_batches": 5,
    "num_inference_steps": 1,
    "action_horizon": 32,
    "stage_order": [
        "preprocess_total_ms",
        "context_get_ms",
        "proprio_context_append_ms",
        "action_latent_init_ms",
        "vae_image_encode_ms",
        "video_pre_dit_ms",
        "attention_mask_build_ms",
        "video_prefill_cache_ms",
        "schedule_build_ms",
        "action_denoise_loop_ms",
        "action_pre_dit_total_ms",
        "action_mot_with_cache_total_ms",
        "action_post_dit_total_ms",
        "scheduler_step_total_ms",
        "action_denormalize_ms",
        "profile_end_to_end_ms"
    ],
    "stage_summary_ms": {
        "mean_preprocess_total_ms": 7.485474803252146,
        "p50_preprocess_total_ms": 7.483125984435901,
        "p95_preprocess_total_ms": 7.505205023335293,
        "mean_context_get_ms": 0.025164399994537234,
        "p50_context_get_ms": 0.024607987143099308,
        "p95_context_get_ms": 0.02745600680103898,
        "mean_proprio_context_append_ms": 0.514910789206624,
        "p50_proprio_context_append_ms": 0.5140979774296284,
        "p95_proprio_context_append_ms": 0.5223540065344423,
        "mean_action_latent_init_ms": 0.1870349980890751,
        "p50_action_latent_init_ms": 0.1872910070233047,

```

```

"p95_action_latent_init_ms": 0.19090698333457112,
"mean_vae_image_encode_ms": 55.2567561971955,
"p50_vae_image_encode_ms": 55.269837990636006,
"p95_vae_image_encode_ms": 55.291980010224506,
"mean_video_pre_dit_ms": 2.574490604456514,
"p50_video_pre_dit_ms": 2.5726989842951298,
"p95_video_pre_dit_ms": 2.5911300035659224,
"mean_attention_mask_build_ms": 0.3458149963989854,
"p50_attention_mask_build_ms": 0.3470630035735667,
"p95_attention_mask_build_ms": 0.35048698191531,
"mean_video_prefill_cache_ms": 115.36812719423324,
"p50_video_prefill_cache_ms": 115.46750998240896,
"p95_video_prefill_cache_ms": 116.06960598146543,
"mean_schedule_build_ms": 0.45045199804008007,
"p50_schedule_build_ms": 0.4514759930316359,
"p95_schedule_build_ms": 0.4610119794961065,
"mean_action_denoise_loop_ms": 115.790387801826,
"p50_action_denoise_loop_ms": 115.30229900381528,
"p95_action_denoise_loop_ms": 116.97550100507215,
"mean_action_pre_dit_total_ms": 1.372487808112055,
"p50_action_pre_dit_total_ms": 1.3641230179928243,
"p95_action_pre_dit_total_ms": 1.3938820047769696,
"mean_action_mot_with_cache_total_ms": 113.97130059776828,
"p50_action_mot_with_cache_total_ms": 113.4979950147681,
"p95_action_mot_with_cache_total_ms": 115.13955099508166,
"mean_action_post_dit_total_ms": 0.1471513940487057,
"p50_action_post_dit_total_ms": 0.1471959985792637,
"p95_action_post_dit_total_ms": 0.15167598030529916,
"mean_scheduler_step_total_ms": 0.11240019230172038,
"p50_scheduler_step_total_ms": 0.11254099081270397,
"p95_scheduler_step_total_ms": 0.11580498539842665,
"mean_action_denormalize_ms": 0.47203239519149065,
"p50_action_denormalize_ms": 0.4655559896491468,
"p95_action_denormalize_ms": 0.4955710028298199,
"mean_profile_end_to_end_ms": 298.9798476046417,
"p50_profile_end_to_end_ms": 298.9027230069041,
"p95_profile_end_to_end_ms": 299.92663199664094
},
"action_step_mean_ms": {
  "pre_dit": 1.372487808112055,
  "mot_with_cache": 113.97130059776828,
  "post_dit": 0.1471513940487057,
  "scheduler": 0.11240019230172038,
  "denoise_loop_total": 115.790387801826
},
"tensor_shapes": {
  "image_tensor": [
    1,
    3,
    384,
    320
  ],
  "proprio": [
    1,
    14
  ],
  "context": [
    1,
    129,
    4096
  ],
  "context_mask": [
    1,
    129
  ],
  "latents_action_initial": [
    1,
    32,
    14
  ],
  "first_frame_latents": [
    1,
    48,
    1,
    24,
    20
  ],
  "video_tokens": [
    1,
    120,
    3072
  ],
  "video_freqs": [
    120,
    1,
    64
  ],
  "video_seq_len": 120,
  "video_tokens_per_frame": 120,
  "attention_mask": [
    152,
    152
  ],
  "video_kv_layers": 30,
  "video_kv0_k": [
    1,
    120,

```

```

    3072
  ],
  "video_kv0_v": [
    1,
    120,
    3072
  ],
  "infer_timesteps_action": [
    1
  ],
  "infer_deltas_action": [
    1
  ],
  "latents_action_final": [
    1,
    32,
    14
  ],
  "action_output": [
    32,
    14
  ]
},
"cuda_memory": {
  "max_memory_allocated_bytes": 13729648128,
  "max_memory_reserved_bytes": 14090764288,
  "memory_allocated_bytes": 13477595136,
  "memory_reserved_bytes": 14090764288
},
"iteration_timings_ms": [
  {
    "preprocess_total_ms": 7.502709981054068,
    "context_get_ms": 0.02553500235080719,
    "proprio_context_append_ms": 0.5223540065344423,
    "action_latent_init_ms": 0.1872910070233047,
    "vae_image_encode_ms": 55.22542301332578,
    "video_pre_dit_ms": 2.5660429964773357,
    "attention_mask_build_ms": 0.3398309927433729,
    "video_prefill_cache_ms": 115.65032199723646,
    "schedule_build_ms": 0.44309202348813415,
    "action_pre_dit_total_ms": 1.3633880007546395,
    "action_mot_with_cache_total_ms": 113.26561699388549,
    "action_post_dit_total_ms": 0.14591700164601207,
    "scheduler_step_total_ms": 0.11580498539842665,
    "action_denoise_loop_ms": 115.07507300120778,
    "action_denormalize_ms": 0.46270800521597266,
    "profile_end_to_end_ms": 298.51207800675184
  },
  {
    "preprocess_total_ms": 7.4728860054165125,
    "context_get_ms": 0.024607987143099308,
    "proprio_context_append_ms": 0.5087859753984958,
    "action_latent_init_ms": 0.18367500160820782,
    "vae_image_encode_ms": 55.291980010224506,
    "video_pre_dit_ms": 2.5534350133966655,
    "attention_mask_build_ms": 0.34802299342118204,
    "video_prefill_cache_ms": 114.52823900617659,
    "schedule_build_ms": 0.4514759930316359,
    "action_pre_dit_total_ms": 1.3858180027455091,
    "action_mot_with_cache_total_ms": 115.13955099508166,
    "action_post_dit_total_ms": 0.1471959985792637,
    "scheduler_step_total_ms": 0.11289300164207816,
    "action_denoise_loop_ms": 116.97550100507215,
    "action_denormalize_ms": 0.4743229947052896,
    "profile_end_to_end_ms": 299.3103280023206
  },
  {
    "preprocess_total_ms": 7.505205023335293,
    "context_get_ms": 0.024479988496750593,
    "proprio_context_append_ms": 0.5140659923199564,
    "action_latent_init_ms": 0.18463499145582318,
    "vae_image_encode_ms": 55.21380697609857,
    "video_pre_dit_ms": 2.5911300035659224,
    "attention_mask_build_ms": 0.35048698191531,
    "video_prefill_cache_ms": 115.12495900387876,
    "schedule_build_ms": 0.45985900214873254,
    "action_pre_dit_total_ms": 1.3552280142903328,
    "action_mot_with_cache_total_ms": 113.4979950147681,
    "action_post_dit_total_ms": 0.15167598030529916,
    "scheduler_step_total_ms": 0.11004498810507357,
    "action_denoise_loop_ms": 115.30229900381528,
    "action_denormalize_ms": 0.465559896491468,
    "profile_end_to_end_ms": 298.24747701059096
  },
  {
    "preprocess_total_ms": 7.483125984435901,
    "context_get_ms": 0.027456000680103898,
    "proprio_context_append_ms": 0.5152499943505973,
    "action_latent_init_ms": 0.1886670070234686,
    "vae_image_encode_ms": 55.269837990636006,
    "video_pre_dit_ms": 2.5891460245475173,
    "attention_mask_build_ms": 0.3470630035735667,
    "video_prefill_cache_ms": 115.46750998240896,
    "schedule_build_ms": 0.4368209920357913,
    "action_pre_dit_total_ms": 1.3938820047769696,
    "action_mot_with_cache_total_ms": 114.74512898712419,
    "action_post_dit_total_ms": 0.14364399248734117,

```



```

    "scheduler_step_total_ms": 0.11071699555031955,
    "action_denoise_loop_ms": 116.57883998122998,
    "action_denormalize_ms": 0.4955710028298199,
    "profile_end_to_end_ms": 299.92663199664094
  },
  {
    "preprocess_total_ms": 7.463447022018954,
    "context_get_ms": 0.023743021301925182,
    "proprio_context_append_ms": 0.5140979774296284,
    "action_latent_init_ms": 0.19090698333457112,
    "vae_image_encode_ms": 55.28273299569264,
    "video_pre_dit_ms": 2.5726989842951298,
    "attention_mask_build_ms": 0.3436710103414953,
    "video_prefill_cache_ms": 116.06960598146543,
    "schedule_build_ms": 0.4610119794961065,
    "action_pre_dit_total_ms": 1.3641230179928243,
    "action_mot_with_cache_total_ms": 113.20821099798195,
    "action_post_dit_total_ms": 0.1473239972256124,
    "scheduler_step_total_ms": 0.11254099081270397,
    "action_denoise_loop_ms": 115.02022601780482,
    "action_denormalize_ms": 0.4620039835572243,
    "profile_end_to_end_ms": 298.9027230069041
  }
],
"partition_notes": [
  "Scheduler is host-side tensor arithmetic and should remain host/CUDA code before forcing it into TensorRT.",
  "The measured action_mot_with_cache stage corresponds to the already exported action denoise-step TensorRT partition core.",
  "VAE image encode and video prefill are one-time per action chunk; action denoise loop scales with num_inference_steps."
]
}

```

15. TensorRT VAE Image Encoder JSON

```

{
  "runtime_mode": "tensorrt_vae_image_encoder",
  "backend": "tensorrt",
  "precision": "fp16_engine",
  "engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/vae_image_encoder_fp16_patched.engine",
  "engine_size_bytes": 318738796,
  "deserialize_ms": 217.50993299065158,
  "warmup_batches": 20,
  "measure_batches": 100,
  "mean_policy_ms": 38.939099922180176,
  "p50_policy_ms": 38.92940902709961,
  "p95_policy_ms": 39.10563278198242,
  "min_policy_ms": 38.72166442871094,
  "max_policy_ms": 39.31913757324219,
  "output_shape": [
    1,
    48,
    1,
    24,
    20
  ],
  "output_finite": true,
  "tensors": [
    {
      "index": 0,
      "name": "input_image",
      "mode": "TensorIOMode.INPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1,
        3,
        384,
        320
      ]
    },
    {
      "index": 1,
      "name": "first_frame_latents",
      "mode": "TensorIOMode.OUTPUT",
      "trt_dtype": "DataType.HALF",
      "torch_dtype": "torch.float16",
      "shape": [
        1,
        48,
        1,
        24,
        20
      ]
    }
  ],
  "note": "This benchmark covers only the TensorRT VAE first-frame image encoder partition, not full FastWAM end-to-end inference."
}

```

16. TensorRT Video Prefill JSON

```

{
  "runtime_mode": "tensorrt_video_prefill",
  "backend": "tensorrt",
  "precision": "fp16_engine",
  "engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/video_prefill_fp16.engine",

```

```

"engine_size_bytes": 9713525884,
"deserialize_ms": 5480.811800982337,
"warmup_batches": 10,
"measure_batches": 50,
"mean_policy_ms": 74.72642440795899,
"p50_policy_ms": 74.7228775024414,
"p95_policy_ms": 74.78707122802734,
"min_policy_ms": 74.66979217529297,
"max_policy_ms": 74.799072265625,
"num_outputs": 60,
"all_outputs_finite": true,
"output_shapes_sample": {
  "video_k_0": [
    1,
    120,
    3072
  ],
  "video_v_0": [
    1,
    120,
    3072
  ],
  "video_k_1": [
    1,
    120,
    3072
  ],
  "video_v_1": [
    1,
    120,
    3072
  ]
},
"note": "This benchmark covers only the TensorRT video prefill/KV cache partition, not full FastWAM end-to-end inference.",
"tensors_omitted_from_report": 63
}

```

17. TensorRT Dynamic-KV Action Step JSON

```

{
  "runtime_mode": "tensorrt_action_step",
  "backend": "tensorrt",
  "precision": "fp16_dynamic_kv_engine",
  "engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_dynamic_kv_fp16.engine",
  "engine_size_bytes": 2052366828,
  "deserialize_ms": 865.587621985469,
  "warmup_batches": 20,
  "measure_batches": 100,
  "mean_policy_ms": 19.031983432769774,
  "p50_policy_ms": 19.031103134155273,
  "p95_policy_ms": 19.051328659057617,
  "min_policy_ms": 19.008224487304688,
  "max_policy_ms": 19.058303833007812,
  "output_shape": [
    1,
    32,
    14
  ],
  "output_finite": true,
  "note": "This benchmark covers only the TensorRT action denoise-step partition, not full FastWAM eager end-to-end inference.",
  "tensors_omitted_from_report": 65
}

```

18. Partitioned TensorRT Runtime JSON

```

{
  "runtime_mode": "partitioned_tensorrt_fastwam",
  "backend": "tensorrt",
  "precision": "fp16_engines_with_fp32_selected_outputs",
  "sample_source": "synthetic_tensor_contract",
  "text_context": "synthetic_zero_context",
  "semantic_text_encoder": "disabled",
  "vae_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/vae_image_encoder_fp16_patched.engine",
  "video_prefill_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/video_prefill_fp16.engine",
  "action_step_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_dynamic_kv_fp16.engine",
  "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
  "warmup_batches": 5,
  "measure_batches": 20,
  "num_inference_steps": 1,
  "action_horizon": 32,
  "action_dim": 14,
  "scheduler": {
    "num_train_timesteps": 1000,
    "shift": 5.0,
    "timestep_dtype": "torch.float16",
    "deltas": [
      -1.0
    ]
  },
  "deserialization_ms": {
    "vae_image_encoder": 168.8888459757436,
    "video_prefill": 4557.959729980212,
    "action_step_dynamic_kv": 1064.211699005682,
    "total": 5791.060274961637
  }
}

```

```

},
"engine_size_bytes": {
  "vae_image_encoder": 318692780,
  "video_prefill": 9713525884,
  "action_step_dynamic_kv": 2052366828
},
"kv_binding": {
  "direct_fp16_bindings": 31,
  "cast_to_fp16_bindings": 29,
  "cast_buffer_names_omitted_from_report": 29
},
"stage_summary_ms": {
  "vae_image_encoder_ms": {
    "mean": 38.950129699707034,
    "p50": 38.935489654541016,
    "p95": 39.06460952758789,
    "min": 38.795616149902344,
    "max": 39.084320068359375
  },
  "video_prefill_ms": {
    "mean": 74.6342170715332,
    "p50": 74.63542175292969,
    "p95": 74.67350769042969,
    "min": 74.60118103027344,
    "max": 74.6785888671875
  },
  "kv_cast_ms": {
    "mean": 0.6114767998456955,
    "p50": 0.6136639714241028,
    "p95": 0.6525120139122009,
    "min": 0.5648959875106812,
    "max": 0.6576640009880066
  },
  "action_denoise_loop_ms": {
    "mean": 19.32935838699341,
    "p50": 19.320512771606445,
    "p95": 19.351327896118164,
    "min": 19.295936584472656,
    "max": 19.478656768798828
  },
  "action_step_engine_total_ms": {
    "mean": 19.07972631454468,
    "p50": 19.077056884765625,
    "p95": 19.10550308227539,
    "min": 19.056127548217773,
    "max": 19.140703201293945
  },
  "scheduler_total_ms": {
    "mean": 0.09048160016536713,
    "p50": 0.08854400366544724,
    "p95": 0.09401600062847137,
    "min": 0.08323200047016144,
    "max": 0.14019200205802917
  },
  "action_decode_ms": {
    "mean": 0.20714399963617325,
    "p50": 0.20147199928760529,
    "p95": 0.22329600155353546,
    "min": 0.19548800587654114,
    "max": 0.28176000714302063
  },
  "end_to_end_ms": {
    "mean": 134.12723735091276,
    "p50": 134.0925129770767,
    "p95": 134.31458300328813,
    "min": 133.93225701292977,
    "max": 134.416960994713
  }
},
"mean_end_to_end_ms": 134.12723735091276,
"p50_end_to_end_ms": 134.0925129770767,
"p95_end_to_end_ms": 134.31458300328813,
"output_shape": [
  1,
  32,
  14
],
"output_finite": true,
"finite_checks": {
  "first_frame_latents": true,
  "video_k_0": true,
  "video_v_0": true,
  "pred_action": true,
  "latents_action": true,
  "action_decoded": true
},
"denormalize_mode": "z_score_global_action_default",
"memory": {
  "before_load": {
    "mem_get_info_free_bytes": 54423248896,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 0,
    "torch_reserved_bytes": 0,
    "torch_max_allocated_bytes": 0,
    "torch_max_reserved_bytes": 0
  },
  "after_vae_load": {

```

```

    "mem_get_info_free_bytes": 54370570240,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 783360,
    "torch_reserved_bytes": 2097152,
    "torch_max_allocated_bytes": 783360,
    "torch_max_reserved_bytes": 2097152
  },
  "after_video_prefill_load": {
    "mem_get_info_free_bytes": 51669020672,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 68577792,
    "torch_reserved_bytes": 96468992,
    "torch_max_allocated_bytes": 68577792,
    "torch_max_reserved_bytes": 96468992
  },
  "after_action_step_load": {
    "mem_get_info_free_bytes": 49413341184,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 113875456,
    "torch_reserved_bytes": 159383552,
    "torch_max_allocated_bytes": 113875456,
    "torch_max_reserved_bytes": 159383552
  },
  "after_benchmark": {
    "mem_get_info_free_bytes": 49124929536,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 90659328,
    "torch_reserved_bytes": 163577856,
    "torch_max_allocated_bytes": 92502528,
    "torch_max_reserved_bytes": 163577856
  },
  "process_ru_maxrss_kib": 19985552
},
"note": "This benchmark loads VAE, video prefill, and dynamic-KV action-step TensorRT engines in one process. It uses synthetic tensors and host-side scheduler/action denormalization; it does not yet include real RoboTwin image preprocessing or text embedding cache.",
"status": "success",
"io_omitted_from_report": true
}

```

19. Real-Sample Partitioned TensorRT Runtime JSON

```

{
  "runtime_mode": "partitioned_tensorrt_fastwam",
  "backend": "tensorrt",
  "precision": "fp16_engines_with_fp32_selected_outputs",
  "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
  "instruction": "Lift the bottle with narrow top head-up from the table",
  "text_context": "synthetic_zero_context",
  "semantic_text_encoder": "disabled",
  "text_cache_path": null,
  "prompt": null,
  "real_input_metadata": {
    "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
    "instruction": "Lift the bottle with narrow top head-up from the table",
    "text_context": "synthetic_zero_context",
    "semantic_text_encoder": "disabled",
    "text_cache_path": null,
    "prompt": null,
    "proprio_source": "extracted_from_checkpoint",
    "proprio_encoder_cache": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/runtime/proprio_encoder.pt",
    "raw_state_shape": [
      14
    ],
    "normalized_proprio_shape": [
      1,
      14
    ]
  },
  "vae_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/vae_image_encoder_fp16_patched.engine",
  "video_prefill_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/video_prefill_fp16.engine",
  "action_step_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_dynamic_kv_fp16.engine",
  "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
  "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
  "warmup_batches": 2,
  "measure_batches": 5,
  "num_inference_steps": 1,
  "action_horizon": 32,
  "action_dim": 14,
  "scheduler": {
    "num_train_timesteps": 1000,
    "shift": 5.0,
    "timestep_dtype": "torch.float16",
    "deltas": [
      -1.0
    ]
  },
  "deserialization_ms": {
    "vae_image_encoder": 1097.3829870054033,
    "video_prefill": 8821.88841799507,
    "action_step_dynamic_kv": 1252.9053619946353,
    "total": 11172.176766995108
  },
  "engine_size_bytes": {

```

```

    "vae_image_encoder": 318692780,
    "video_prefill": 9713525884,
    "action_step_dynamic_kv": 2052366828
  },
  "kv_binding": {
    "direct_fp16_bindings": 31,
    "cast_to_fp16_bindings": 29,
    "cast_buffer_names_omitted_from_report": 29
  },
  "stage_summary_ms": {
    "vae_image_encoder_ms": {
      "mean": 39.17632064819336,
      "p50": 39.210113525390625,
      "p95": 39.260799407958984,
      "min": 39.02000045776367,
      "max": 39.260799407958984
    },
    "video_prefill_ms": {
      "mean": 74.94142456054688,
      "p50": 74.94457244873047,
      "p95": 74.9537582397461,
      "min": 74.91471862792969,
      "max": 74.9537582397461
    },
    "kv_cast_ms": {
      "mean": 0.5757311940193176,
      "p50": 0.5741440057754517,
      "p95": 0.593887984752655,
      "min": 0.5509759783744812,
      "max": 0.593887984752655
    },
    "action_denoise_loop_ms": {
      "mean": 19.351942443847655,
      "p50": 19.352703094482422,
      "p95": 19.36729621887207,
      "min": 19.338176727294922,
      "max": 19.36729621887207
    },
    "action_step_engine_total_ms": {
      "mean": 19.102579879760743,
      "p50": 19.103776931762695,
      "p95": 19.11280059814453,
      "min": 19.094432830810547,
      "max": 19.11280059814453
    },
    "scheduler_total_ms": {
      "mean": 0.08982399851083755,
      "p50": 0.0888959988951683,
      "p95": 0.09638399630784988,
      "min": 0.08620800077915192,
      "max": 0.09638399630784988
    },
    "action_decode_ms": {
      "mean": 0.2091327965259552,
      "p50": 0.20950399339199066,
      "p95": 0.23164799809455872,
      "min": 0.1945919990539508,
      "max": 0.23164799809455872
    },
    "end_to_end_ms": {
      "mean": 134.67669179663062,
      "p50": 134.6913370070979,
      "p95": 134.76467097643763,
      "min": 134.60286799818277,
      "max": 134.76467097643763
    }
  },
  "mean_end_to_end_ms": 134.67669179663062,
  "p50_end_to_end_ms": 134.6913370070979,
  "p95_end_to_end_ms": 134.76467097643763,
  "output_shape": [
    1,
    32,
    14
  ],
  "output_finite": true,
  "finite_checks": {
    "first_frame_latents": true,
    "video_k_0": true,
    "video_v_0": true,
    "pred_action": true,
    "latents_action": true,
    "action_decoded": true
  },
  "denormalize_mode": "z_score_global_action_default",
  "memory": {
    "before_load": {
      "mem_get_info_free_bytes": 54391357440,
      "mem_get_info_total_bytes": 65880092672,
      "torch_allocated_bytes": 0,
      "torch_reserved_bytes": 0,
      "torch_max_allocated_bytes": 0,
      "torch_max_reserved_bytes": 0
    },
    "after_vae_load": {
      "mem_get_info_free_bytes": 54336102400,
      "mem_get_info_total_bytes": 65880092672,

```

```

    "torch_allocated_bytes": 783360,
    "torch_reserved_bytes": 2097152,
    "torch_max_allocated_bytes": 783360,
    "torch_max_reserved_bytes": 2097152
  },
  "after_video_prefill_load": {
    "mem_get_info_free_bytes": 52604579840,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 68577792,
    "torch_reserved_bytes": 96468992,
    "torch_max_allocated_bytes": 68577792,
    "torch_max_reserved_bytes": 96468992
  },
  "after_action_step_load": {
    "mem_get_info_free_bytes": 50345959424,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 113875456,
    "torch_reserved_bytes": 159383552,
    "torch_max_allocated_bytes": 113875456,
    "torch_max_reserved_bytes": 159383552
  },
  "after_benchmark": {
    "mem_get_info_free_bytes": 49679671296,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 100227584,
    "torch_reserved_bytes": 165675008,
    "torch_max_allocated_bytes": 102070784,
    "torch_max_reserved_bytes": 165675008
  },
  "process_ru_maxrss_kib": 25051328
},
"note": "This benchmark loads VAE, video prefill, and dynamic-KV action-step TensorRT engines in one process. Scheduler and action denormalization remain host-side. When --sample is set, image preprocessing and state/proprio-token construction use a real offline RoboTwin sample; semantic text still depends on whether --text-cache-dir is provided.",
"status": "success",
"io_omitted_from_report": true
}

```

20. Real-Sample + Text Cache Partitioned TensorRT Runtime JSON

```

{
  "runtime_mode": "partitioned_tensorrt_fastwam",
  "backend": "tensorrt",
  "precision": "fp16_engines_with_fp32_selected_outputs",
  "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
  "instruction": "Lift the bottle with narrow top head-up from the table",
  "text_context": "precomputed_t5_cache",
  "semantic_text_encoder": "precomputed_cache",
  "text_cache_path": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/text_embeds_cache/robotwin/82243a4317e52ea0809eaaef4ab867833e575409d75cc0eabddd4d352133bde2.t5_len128.wan22ti2v5b.pt",
  "prompt": "A video recorded from a robot's point of view executing the following instruction: Lift the bottle with narrow top head-up from the table",
  "real_input_metadata": {
    "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
    "instruction": "Lift the bottle with narrow top head-up from the table",
    "text_context": "precomputed_t5_cache",
    "semantic_text_encoder": "precomputed_cache",
    "text_cache_path": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/data/text_embeds_cache/robotwin/82243a4317e52ea0809eaaef4ab867833e575409d75cc0eabddd4d352133bde2.t5_len128.wan22ti2v5b.pt",
    "prompt": "A video recorded from a robot's point of view executing the following instruction: Lift the bottle with narrow top head-up from the table",
    "proprio_source": "cached_proprio_encoder",
    "proprio_encoder_cache": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/runtime/proprio_encoder.pt",
    "raw_state_shape": [
      14
    ],
    "normalized_proprio_shape": [
      1,
      14
    ]
  },
  "vae_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/vae_image_encoder_fp16_patched.engine",
  "video_prefill_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/video_prefill_fp16.engine",
  "action_step_engine": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_dynamic_kv_fp16.engine",
  "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
  "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
  "warmup_batches": 2,
  "measure_batches": 5,
  "num_inference_steps": 1,
  "action_horizon": 32,
  "action_dim": 14,
  "scheduler": {
    "num_train_timesteps": 1000,
    "shift": 5.0,
    "timestep_dtype": "torch.float16",
    "deltas": [
      -1.0
    ]
  },
  "deserialization_ms": {
    "vae_image_encoder": 212.92611700482666,
    "video_prefill": 40606.328980007675,
    "action_step_dynamic_kv": 2349.550293991342,

```

```

"total": 43168.80539100384
},
"engine_size_bytes": {
  "vae_image_encoder": 318738796,
  "video_prefill": 9713525884,
  "action_step_dynamic_kv": 2052366828
},
"kv_binding": {
  "direct_fp16_bindings": 31,
  "cast_to_fp16_bindings": 29,
  "cast_buffer_names_omitted_from_report": 29
},
"stage_summary_ms": {
  "vae_image_encoder_ms": {
    "mean": 39.35005340576172,
    "p50": 39.31020736694336,
    "p95": 39.56047821044922,
    "min": 39.25859069824219,
    "max": 39.56047821044922
  },
  "video_prefill_ms": {
    "mean": 74.64688720703126,
    "p50": 74.64985656738281,
    "p95": 74.6632308959961,
    "min": 74.62643432617188,
    "max": 74.6632308959961
  },
  "kv_cast_ms": {
    "mean": 0.547110402584076,
    "p50": 0.5275840163230896,
    "p95": 0.6144959926605225,
    "min": 0.5120319724082947,
    "max": 0.6144959926605225
  },
  "action_denoise_loop_ms": {
    "mean": 19.38736000061035,
    "p50": 19.380735397338867,
    "p95": 19.41391944885254,
    "min": 19.36457633972168,
    "max": 19.41391944885254
  },
  "action_step_engine_total_ms": {
    "mean": 19.136915969848634,
    "p50": 19.131040573120117,
    "p95": 19.165664672851562,
    "min": 19.122880935668945,
    "max": 19.165664672851562
  },
  "scheduler_total_ms": {
    "mean": 0.09052160084247589,
    "p50": 0.09008000046014786,
    "p95": 0.09241600334644318,
    "min": 0.08931200206279755,
    "max": 0.09241600334644318
  },
  "action_decode_ms": {
    "mean": 0.21153919994831086,
    "p50": 0.21145600080490112,
    "p95": 0.21743999421596527,
    "min": 0.20774400234222412,
    "max": 0.21743999421596527
  },
  "end_to_end_ms": {
    "mean": 134.57489520078525,
    "p50": 134.53775600646622,
    "p95": 134.6947140118573,
    "min": 134.43001598352566,
    "max": 134.6947140118573
  }
},
"mean_end_to_end_ms": 134.57489520078525,
"p50_end_to_end_ms": 134.53775600646622,
"p95_end_to_end_ms": 134.6947140118573,
"output_shape": [
  1,
  32,
  14
],
"output_finite": true,
"finite_checks": {
  "first_frame_latents": true,
  "video_k_0": true,
  "video_v_0": true,
  "pred_action": true,
  "latents_action": true,
  "action_decoded": true
},
"denormalize_mode": "z_score_global_action_default",
"memory": {
  "before_load": {
    "mem_get_info_free_bytes": 54870859776,
    "mem_get_info_total_bytes": 65880092672,
    "torch_allocated_bytes": 0,
    "torch_reserved_bytes": 0,
    "torch_max_allocated_bytes": 0,
    "torch_max_reserved_bytes": 0
  },

```

```

"after_vae_load": {
  "mem_get_info_free_bytes": 54818435072,
  "mem_get_info_total_bytes": 65880092672,
  "torch_allocated_bytes": 783360,
  "torch_reserved_bytes": 2097152,
  "torch_max_allocated_bytes": 783360,
  "torch_max_reserved_bytes": 2097152
},
"after_video_prefill_load": {
  "mem_get_info_free_bytes": 51684290560,
  "mem_get_info_total_bytes": 65880092672,
  "torch_allocated_bytes": 68577792,
  "torch_reserved_bytes": 96468992,
  "torch_max_allocated_bytes": 68577792,
  "torch_max_reserved_bytes": 96468992
},
"after_action_step_load": {
  "mem_get_info_free_bytes": 49428885504,
  "mem_get_info_total_bytes": 65880092672,
  "torch_allocated_bytes": 113875456,
  "torch_reserved_bytes": 159383552,
  "torch_max_allocated_bytes": 113875456,
  "torch_max_reserved_bytes": 159383552
},
"after_benchmark": {
  "mem_get_info_free_bytes": 48785362944,
  "mem_get_info_total_bytes": 65880092672,
  "torch_allocated_bytes": 100227584,
  "torch_reserved_bytes": 165675008,
  "torch_max_allocated_bytes": 102070784,
  "torch_max_reserved_bytes": 165675008
},
"process_ru_maxrss_kib": 19989792
},
"note": "This benchmark loads VAE, video prefill, and dynamic-KV action-step TensorRT engines in one process. Scheduler and action
denormalization remain host-side. When --sample is set, image preprocessing and state/proprio-token construction use a real offline RoboTwin
sample; semantic text still depends on whether --text-cache-dir is provided.",
"status": "success",
"io_omitted_from_report": true
}

```

21. Eager-vs-TRT Drift Validation JSON

```

{
  "status": "success",
  "validation_mode": "eager_fp16_reference_vs_partitioned_trt_fp16_real_robotwin_text_cache",
  "preset": "robotwin_uncond_3cam_384",
  "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-
deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
  "instruction": "Lift the bottle with narrow top head-up from the table",
  "text_context": "precomputed_t5_cache",
  "num_inference_steps": 1,
  "action_horizon": 32,
  "seed": 1234,
  "selected_kv_layers": [
    0,
    15,
    29
  ],
  "eager_metadata": {
    "runtime_mode": "eager_reference",
    "backend": "pytorch",
    "mixed_precision": "fp16",
    "checkpoint": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384.pt",
    "dataset_stats": "/root/wujinlong/nvme/tinyaction_fastwam/artifacts/checkpoints/fastwam_release/robotwin_uncond_3cam_384_dataset_stats.json",
    "load_ms": 67104.52098899987,
    "run_ms": 795.8467220014427,
    "text_context": {
      "text_context": "precomputed_t5_cache",
      "text_cache_path": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-
deploy/data/text_embeddings_cache/robotwin/82243a4317e52ea0809eaaef4ab867833e575409d75cc0eabddd4d352133bde2.t5_len128.wan22ti2v5b.pt",
      "semantic_text_encoder": "precomputed_cache",
      "prompt": "A video recorded from a robot's point of view executing the following instruction: Lift the bottle with narrow top head-up from
the table"
    },
    "video_seq_len": 120,
    "selected_kv_layers": [
      0,
      15,
      29
    ]
  },
  "trt_metadata": {
    "runtime_mode": "partitioned_tensorrt_candidate",
    "backend": "tensorrt",
    "precision": "fp16_engines",
    "load_ms": 11868.24805699871,
    "run_ms": 173.6860869859811,
    "engines": {
      "vae": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/vae_image_encoder_fp16_patched.engine",
      "video_prefill": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/video_prefill_fp16.engine",
      "action_step": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/trt/action_step_dynamic_kv_fp16.engine"
    },
    "engine_size_bytes": {
      "vae": 318738796,

```



```

    "video_prefill": 9713525884,
    "action_step": 2052366828
  },
  "real_input_metadata": {
    "sample_source": "offline_sample:/root/wujinlong/nvme/tinyaction_fastwam/fastwam-
deploy/data/real_samples/robotwin_unified_ep000000_frame000000.npz",
    "instruction": "Lift the bottle with narrow top head-up from the table",
    "text_context": "precomputed_t5_cache",
    "semantic_text_encoder": "precomputed_cache",
    "text_cache_path": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-
deploy/data/text_embeds_cache/robotwin/82243a4317e52ea0809eaaef4ab867833e575409d75cc0eabddd4d352133bde2.t5_len128.wan22ti2v5b.pt",
    "prompt": "A video recorded from a robot's point of view executing the following instruction: Lift the bottle with narrow top head-up from
the table",
    "proprio_source": "cached_proprio_encoder",
    "proprio_encoder_cache": "/root/wujinlong/nvme/tinyaction_fastwam/fastwam-deploy/outputs/runtime/proprio_encoder.pt",
    "raw_state_shape": [
      14
    ],
    "normalized_proprio_shape": [
      1,
      14
    ]
  },
  "denormalize_mode": "z_score_global_action_default",
  "kv_binding": {
    "direct_fp16_bindings": 31,
    "cast_to_fp16_bindings": 29,
    "cast_buffer_names": [
      "video_v_1",
      "video_v_10",
      "video_v_11",
      "video_v_12",
      "video_v_13",
      "video_v_14",
      "video_v_15",
      "video_v_16",
      "video_v_17",
      "video_v_18",
      "video_v_19",
      "video_v_2",
      "video_v_20",
      "video_v_21",
      "video_v_22",
      "video_v_23",
      "video_v_24",
      "video_v_25",
      "video_v_26",
      "video_v_27",
      "video_v_28",
      "video_v_29",
      "video_v_3",
      "video_v_4",
      "video_v_5",
      "video_v_6",
      "video_v_7",
      "video_v_8",
      "video_v_9"
    ]
  },
  "memory_free_bytes": {
    "before_load": 48807645184,
    "after_run": 44256215040
  }
},
"comparisons": {
  "context_with_proprio": {
    "status": "compared",
    "shape": [
      1,
      129,
      4096
    ],
    "reference_finite": true,
    "candidate_finite": true,
    "max_abs": 0.0,
    "mean_abs": 0.0,
    "rmse": 0.0,
    "max_rel": 0.0,
    "mean_rel": 0.0,
    "cosine": 0.999993547807746
  },
  "latents_initial": {
    "status": "compared",
    "shape": [
      1,
      32,
      14
    ],
    "reference_finite": true,
    "candidate_finite": true,
    "max_abs": 0.0,
    "mean_abs": 0.0,
    "rmse": 0.0,
    "max_rel": 0.0,
    "mean_rel": 0.0,
    "cosine": 0.999998745159722
  }
},

```

```

"first_frame_latents": {
  "status": "compared",
  "shape": [
    1,
    48,
    1,
    24,
    20
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 4.00390625,
  "mean_abs": 0.7252460718154907,
  "rmse": 0.9060947299003601,
  "max_rel": 2.0,
  "mean_rel": 0.7129542231559753,
  "cosine": 0.59732355067508
},
"pred_action_first": {
  "status": "compared",
  "shape": [
    1,
    32,
    14
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 2.19140625,
  "mean_abs": 0.39483293890953064,
  "rmse": 0.6724507808685303,
  "max_rel": 1.9802581071853638,
  "mean_rel": 0.4128654897212982,
  "cosine": 0.8498515885339047
},
"pred_action_last": {
  "status": "compared",
  "shape": [
    1,
    32,
    14
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 2.19140625,
  "mean_abs": 0.39483293890953064,
  "rmse": 0.6724507808685303,
  "max_rel": 1.9802581071853638,
  "mean_rel": 0.4128654897212982,
  "cosine": 0.8498515885339047
},
"latents_action_final": {
  "status": "compared",
  "shape": [
    1,
    32,
    14
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 2.19140625,
  "mean_abs": 0.3948325514793396,
  "rmse": 0.6724430322647095,
  "max_rel": 1.9463220834732056,
  "mean_rel": 0.487869530916214,
  "cosine": 0.5516912002954459
},
"action_decoded": {
  "status": "compared",
  "shape": [
    1,
    32,
    14
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 1.598406434059143,
  "mean_abs": 0.2771923243999481,
  "rmse": 0.4767465889453888,
  "max_rel": 1.994829773902893,
  "mean_rel": 0.7988913655281067,
  "cosine": 0.7157925372223637
},
"schedule_timesteps": {
  "status": "compared",
  "shape": [
    1
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 0.0,
  "mean_abs": 0.0,
  "rmse": 0.0,
  "max_rel": 0.0,
  "mean_rel": 0.0,
  "cosine": 1.0
},
"schedule_deltas": {

```

```
"status": "compared",
"shape": [
  1,
],
"reference_finite": true,
"candidate_finite": true,
"max_abs": 0.0,
"mean_abs": 0.0,
"rmse": 0.0,
"max_rel": 0.0,
"mean_rel": 0.0,
"cosine": 1.0
},
"video_k_0": {
  "status": "compared",
  "shape": [
    1,
    120,
    3072
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 16.78125,
  "mean_abs": 0.5575255155563354,
  "rmse": 1.323319911956787,
  "max_rel": 2.0,
  "mean_rel": 0.8460207581520081,
  "cosine": 0.2579873440762585
},
"video_v_0": {
  "status": "compared",
  "shape": [
    1,
    120,
    3072
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 6.853515625,
  "mean_abs": 0.5651108026504517,
  "rmse": 0.7817215919494629,
  "max_rel": 2.0,
  "mean_rel": 0.8670796751976013,
  "cosine": 0.43662697080547885
},
"video_k_15": {
  "status": "compared",
  "shape": [
    1,
    120,
    3072
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 20.765625,
  "mean_abs": 1.2359435558319092,
  "rmse": 1.6813918352127075,
  "max_rel": 2.0,
  "mean_rel": 0.8895217180252075,
  "cosine": 0.28377176818604505
},
"video_v_15": {
  "status": "compared",
  "shape": [
    1,
    120,
    3072
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 9.119140625,
  "mean_abs": 1.1384261846542358,
  "rmse": 1.4679687023162842,
  "max_rel": 2.0,
  "mean_rel": 0.9722861647605896,
  "cosine": 0.058229624879573616
},
"video_k_29": {
  "status": "compared",
  "shape": [
    1,
    120,
    3072
  ],
  "reference_finite": true,
  "candidate_finite": true,
  "max_abs": 115.62890625,
  "mean_abs": 1.0279563665390015,
  "rmse": 3.461761713027954,
  "max_rel": 2.0,
  "mean_rel": 0.8572744727134705,
  "cosine": 0.6652239662760239
},
"video_v_29": {
  "status": "compared",
  "shape": [
    1,
```

```
120,
3072
],
"reference_finite": true,
"candidate_finite": true,
"max_abs": 4.05419921875,
"mean_abs": 0.4849422574043274,
"rmse": 0.6272431015968323,
"max_rel": 2.0,
"mean_rel": 0.9411901831626892,
"cosine": 0.19370896831722675
}
},
"summary": {
  "num_compared_tensors": 15,
  "worst_max_abs": 115.62890625,
  "all_compared_finite": true,
  "note": "This is a numerical drift report, not a simulator success-rate report. TensorRT FP16 engines are compared against an eager FP16 reference on the same real RoboTwin sample, real T5 text cache, seed, initial action latent, and scheduler."
},
"tensor_summaries_omitted_from_report": true
}
```

22. 后续阶段

ID	任务	说明
N001	Maintain real FastWAM text embedding cache flow	The first real RoboTwin instruction cache has been generated and used. For new RoboTwin tasks, repeat the upstream precompute_text_embeds.py flow and copy only the prompt-specific .pt entries needed for AGX inference.
N002	Investigate TensorRT numerical drift before simulator validation	Drift validation now runs and all outputs are finite, but selected video K/V and action outputs show non-trivial FP16 drift. Next step is to localize whether VAE export, video prefill FP16 normalization, or scheduler/action precision dominates the final action drift.
N003	Design a real calibrated INT8 path	Representative RoboTwin calibration is still required. The current synthetic/uncalibrated INT8 artifacts show speed potential but are not accuracy-valid, and video/action calibrated builds need graph splitting or output-pruned calibration to avoid OOM/illegal memory access.
N004	Keep disk cleanup manifest current	The first FastWAM internal cleanup recovered required space and removed obsolete artifacts. Before more large builds or new checkpoints, re-run a manifest review and delete only artifacts that are no longer required by the current FP16 runtime chain.

TensorRT 阶段会继续追加记录依赖版本、ONNX/engine 转换、算子不兼容、量化校准、CUDA runtime 推理等问题。